

Sistemas de Cadastro Acadêmico: Comparação de Desempenho com AVL e BB

Alef Cauan Sousa Rodrigues

Abstract

O projeto envolve o desenvolvimento de um programa em C para gerenciar dados sobre alunos e seus históricos acadêmicos em cursos. As estruturas de dados usadas incluem listas e árvores, organizadas conforme a necessidade de cada entidade: alunos, cursos, disciplinas, notas e matrículas.

Além disso, foi feita a avaliação de desempenho. O tempo de inserção de cursos e de busca de notas foi medido utilizando a função `time` da linguagem C, sendo que essas operações foram realizadas múltiplas vezes para cálculo da média. Na terceira parte do projeto, todas as funcionalidades foram reimplementadas usando árvores AVL.

1 Introdução

O projeto desenvolvido explora a implementação de estruturas de dados eficientes, focando na organização e gerenciamento de informações acadêmicas, como alunos, cursos, disciplinas, matrículas e notas. Estruturas de dados são essenciais na Ciência da Computação, pois definem maneiras otimizadas de armazenar e organizar dados, permitindo que os sistemas sejam mais rápidos e escaláveis. Neste contexto, as árvores binárias se destacam como uma solução versátil, amplamente utilizada em sistemas de arquivos, bancos de dados e compiladores.

O trabalho tem como foco a comparação entre duas variações de árvores: a Árvore Binária de

Busca (BB) e a Árvore AVL. A árvore BB oferece uma estrutura simples para organização de dados, mas pode se tornar ineficiente caso não esteja balanceada. A AVL, por sua vez, é uma versão auto-balanceada da árvore BB, garantindo melhor desempenho nas operações de inserção, remoção e busca em cenários com grandes volumes de dados.

Além de abordar as características dessas duas estruturas, o projeto desenvolve um sistema universitário básico em C, utilizando tanto árvores BB quanto AVL para armazenar e organizar dados como cursos, disciplinas e matrículas. O objetivo é permitir operações de cadastro e consulta, garantindo a integridade e eficiência do sistema. As operações incluem inserção e remoção de alunos, cadastro de notas e exibição de históricos.

Por fim, o projeto também realiza uma análise de desempenho, comparando o tempo de execução das operações de inserção e busca nas árvores BB e AVL. Serão realizados testes com diferentes cenários, visando identificar as situações em que a AVL, com seu balanceamento automático, oferece uma vantagem significativa em termos de tempo de execução em relação à árvore BB. Assim, o estudo busca demonstrar a importância de escolher a estrutura de dados adequada para garantir sistemas eficientes e escaláveis.

2 Seções Específicas

Nesta seção, serão abordadas as principais funções que compõem o sistema acadêmico desenvolvido, assim como a organização das estruturas utilizadas. O foco será nos processos fundamentais que garantem o bom funcionamento do sistema, desde o cadastro e armazenamento dos dados até o gerenciamento eficiente das árvores de dados, como as Árvores de Busca Binária (BB) e Árvores AVL.

Cada função desempenha um papel específico na construção e manutenção das estruturas necessárias, com ênfase nas operações de inserção, remoção, e busca de dados. A implementação de algoritmos que garantem a integridade e o balanceamento das árvores será discutida, além de

e abordagens computacionais utilizadas para otimizar o desempenho, com foco em como as Árvores AVL são empregadas para garantir o balanceamento dinâmico das estruturas, resultando em tempos de acesso mais rápidos e consistentes. Essa organização visa proporcionar ao leitor uma compreensão abrangente do funcionamento interno do sistema, estabelecendo as bases para as subseções onde as funcionalidades específicas serão discutidas em maior detalhe.

2.1 Aleatoriedade

Para gerar a aleatoriedade e realizar os testes com as árvores BB e AVL na linguagem C, utilizou-se a biblioteca ‘time.h’. Esta biblioteca é fundamental para trabalhar com funções relacionadas ao tempo no C, e seu uso foi especialmente importante para inicializar a semente do gerador de números aleatórios, através da função ‘srand(time(NULL))’. Ao utilizar o valor retornado por ‘time(NULL)’, que fornece o número de segundos desde uma data de referência (normalmente 1º de janeiro de 1970), garante-se que a sequência de números gerada seja diferente a cada execução do programa. Isso é crucial para simular diferentes cenários e gerar uma aleatoriedade confiável nos testes, permitindo comparar o comportamento das duas estruturas de árvore com diferentes padrões de inserção e busca.

2.2 Sobre as Estruturas

Neste seção, serão apresentadas as estruturas que foram utilizadas e suas características gerais. Além disso, um breve contexto de onde foram utilizadas.

2.2.1 Lista Encadeada

A lista encadeada é uma estrutura de dados linear onde cada elemento, chamado de nó, armazena um valor e um ponteiro para o próximo nó da lista. Diferente de um vetor, a lista encadeada não exige um bloco de memória contíguo, tornando-a eficiente para inserções e remoções dinâmicas, principalmente no início e no meio da lista. No contexto deste trabalho, a lista encadeada é usada para organizar os alunos por ordem alfabética, facilitando a busca e a manutenção dos dados. A flexibilidade dessa estrutura permite adicionar ou remover alunos sem necessidade de realocar ou reorganizar a memória, sendo ideal para o cadastro contínuo de novos alunos.

2.2.2 Árvore Binária

A árvore binária é uma estrutura hierárquica em que cada nó possui, no máximo, dois filhos, geralmente chamados de filho esquerdo e filho direito. Essa organização permite uma divisão eficiente dos dados em diferentes níveis, sendo particularmente útil para representações de hierarquias ou relações recursivas. Cada nó de uma árvore binária contém um valor e referências para seus filhos. Embora simples, a árvore binária não garante eficiência de busca ou balanceamento, o que pode resultar em um desempenho degradado em situações onde os nós são inseridos de forma desbalanceada.

2.2.3 Árvore Binária de Busca

A árvore binária de busca (BB) é uma variação da árvore binária, onde os nós são organizados de forma que para qualquer nó, todos os valores à esquerda são menores e todos os valores à direita são maiores. Essa propriedade facilita a realização de operações de busca, inserção e remoção de forma mais eficiente que em uma árvore binária comum, já que os dados são ordenados naturalmente. No sistema em questão, a BB é utilizada para organizar informações como cursos, disciplinas, e notas dos alunos, permitindo a recuperação rápida de dados com base em critérios específicos como código ou nome. No entanto, a BB pode se desbalancear, o que impacta negativamente o tempo de execução das operações.

2.2.4 Árvore AVL

A árvore AVL é uma extensão da árvore binária de busca que mantém seu balanceamento de forma automática. Após cada inserção ou remoção, a árvore é reajustada por meio de rotações, garantindo que a diferença de altura entre as subárvores esquerda e direita de qualquer nó não exceda 1. Isso proporciona operações de busca, inserção e remoção com complexidade logarítmica, mesmo nos piores cenários. A AVL é essencial para garantir que o sistema mantenha eficiência ao lidar com grandes volumes de dados, como o cadastro de disciplinas e a manipulação de matrículas e notas dos alunos. O balanceamento automático das árvores AVL torna essa estrutura especialmente útil em sistemas onde o acesso rápido e ordenado aos dados é uma prioridade.

2.3 Informações Técnicas

PARÂMETRO	DESCRIÇÃO
Sistema Operacional	Ubuntu 22.04
Processador	Intel i3-1115G4 3.0 GHz
Memória RAM	8 GB DDR4

Table 1: Especificações do Sistema

2.4 Estruturas de Lista

A seguir estão as estruturas de lista utilizadas, estão presentes apenas seus detalhes básicos, sem as características específicas necessárias para funcionar no programa.

2.4.1 Estrutura de Estudante

```
typedef struct Student {
    ///////////////
    struct Student *next;
} Student;
```

Listing 1: Student

2.4.2 Estrutura Lista de Estudante

```
typedef struct Student_list {
    Student *first;
} Student_list;
```

Listing 2: Studentlist

2.5 Estruturas de Árvore de Busca

A seguir estão as estruturas de Árvore utilizadas, estão presentes apenas seus detalhes básicos, sem as características específicas necessárias para funcionar no programa, que engloba a diferença da árvore BB para AVL, ou seja, a altura da árvore.

2.5.1 Estrutura de Curso

```
typedef struct Course {
    ///////////////
    struct Course *left;
    struct Course *right;
} Course;
```

Listing 3: Course

2.5.2 Árvore de Curso

```
typedef struct Course_tree {
    Course *root;
} Course_tree;
```

Listing 4: CourseTree

2.5.3 Estrutura de Disciplina

```
typedef struct Discipline {
    ///////////////
    struct Discipline *left;
    struct Discipline *right;
} Discipline;
```

Listing 5: Discipline

2.5.4 Árvore de Disciplina

```
typedef struct Discipline_tree {
    Discipline *root;
} Discipline_tree;
```

Listing 6: DisciplineTree

2.5.5 Estrutura de Notas

```
typedef struct Grade {
    ///////////////
    struct Grade *left;
    struct Grade *right;
} Grade;
```

Listing 7: Grade

2.5.6 Árvore de Notas

```
typedef struct Grade_tree {
    Grade *root;
} Grade_tree;
```

Listing 8: GradeTree

2.5.7 Estrutura de Matricula

```
typedef struct Enrollment {
    ///////////////
    struct Enrollment *left;
    struct Enrollment *right;
} Enrollment;
```

Listing 9: Enrollment

2.5.8 Árvore de Matricula

```
typedef struct Enrollment_tree {
    Enrollment *root;
} Enrollment_tree;
```

Listing 10: EnrollmentTree

2.6 Funções Principais

Nesta seção, são apresentadas as principais funções do sistema, que operam sobre as estruturas de dados representando alunos, cursos, disciplinas, matrículas e notas. As funções são responsáveis por cadastrar, remover e exibir informações, garantindo a consistência dos dados. Abaixo, as funções são descritas de forma resumida, com ênfase nas operações que realizam.

2.7 Funções Principais

2.7.1 Cadastrar Alunos

Esta função permite o cadastro de novos estudantes, garantindo que o código do curso associado já esteja cadastrado na árvore de cursos.

```
void register_student(Student_list *  
list, Course *courses);
```

2.7.2 Cadastrar Cursos

A função cadastra novos cursos na árvore de cursos, sem a necessidade de disciplinas previamente cadastradas. Ela insere o novo curso de forma balanceada na árvore AVL.

```
void register_course(Course **root);
```

2.7.3 Cadastrar Disciplinas

Permite cadastrar disciplinas para um curso existente, verificando a validade do período (entre 1 e o número máximo de períodos do curso) e se a carga horária é múltiplo de 15.

```
void register_discipline(Discipline  
**root, Course *course);
```

2.7.4 Cadastrar Matrícula

Cadastra uma nova matrícula para o aluno em uma disciplina específica, organizando-as em uma árvore AVL.

```
void register_enrollment(Enrollment  
**root, int discipline_code);
```

2.7.5 Cadastrar Notas

Cadastra notas para disciplinas em que o aluno está matriculado. Após o cadastro da nota, a disciplina é movida da árvore de matrículas para a árvore de notas.

```
void register_grade(Student **  
student);
```

2.7.6 Mostrar Alunos

Exibe todos os alunos matriculados em um curso específico.

```
void show_students_by_course(Student  
_list *list, int course_code);
```

2.7.7 Mostrar Cursos

Exibe todos os cursos cadastrados na árvore de cursos.

```
void show_courses(Course *root);
```

2.7.8 Mostrar Disciplinas

Exibe todas as disciplinas de um curso específico, organizadas em sua árvore AVL.

```
void show_disciplines(Discipline *  
root);
```

2.7.9 Mostrar Disciplinas no Período

Exibe todas as disciplinas cadastradas em um determinado período de um curso.

```
void show_disciplines_by_period(  
Discipline *root, int period);
```

2.7.10 Mostrar Disciplinas Matriculadas

Exibe as disciplinas em que um aluno está atualmente matriculado.

```
void show_enrolled_disciplines(  
Enrollment *root, Discipline *  
disc_root);
```

2.7.11 Mostrar Notas

Exibe as notas de disciplinas de um aluno, filtrando por período.

```
void show_grades(Student *student,  
Discipline *root, int period);
```

2.7.12 Mostrar Notas no Período

Exibe todas as notas de um aluno em um determinado período, incluindo a carga horária das disciplinas.

```
void show_period_grade(Student *  
student, Discipline *discipline);
```

2.7.13 Remover Disciplina

Remove uma disciplina de um curso, desde que não tenha alunos matriculados na mesma.

```
bool remove_discipline(Discipline **
    root, Student *head, int
    discipline_code);
```

2.7.14 Remover Matrícula

Remove a matrícula de um aluno em uma disciplina, removendo-a da árvore de matrículas.

```
bool remove_enrollment(Enrollment **
    head, int discipline_code);
```

2.7.15 Mostrar Histórico

Exibe o histórico acadêmico de um aluno, organizando as disciplinas por período.

```
void show_history(Student *student,
    Discipline *root, int period);
```

3 Resultados da Execução do Programa

Para avaliar a performance das Árvores Binárias de Busca (BB) e das Árvores AVL, realizamos testes com inserções e buscas em três cenários distintos: valores em ordem crescente, decrescente e aleatória. Esses testes foram realizados com árvores já povoadas com 10.000 elementos. A seguir, detalhamos os resultados.

3.1 Inserções na Árvore Binária de Busca (BB)

Na Tabela 2, são apresentados os tempos de inserção da Árvore Binária de Busca (BB) nos três cenários: crescente, decrescente e aleatório. Observa-se que nos cenários crescente e decrescente, a árvore tem um desempenho significativamente inferior, com tempos que variam de 83.0 a 102.0. Isso ocorre porque a árvore se degenera em uma lista nesses cenários, o que resulta em uma maior profundidade e, consequentemente, em inserções mais lentas. Em contrapartida, no cenário aleatório, a árvore apresenta tempos bem menores, próximos de 0.0, o que se deve à menor probabilidade de desequilíbrio, tornando as inserções mais eficientes.

3.2 Buscas na Árvore Binária de Busca (BB)

Os resultados das buscas na Árvore BB, detalhados na Tabela 3, mostram um comportamento

BB INSERÇÃO		
CRESC.	DECRES	RANDOM
89.0	83.0	0.0
88.0	83.0	1.0
90.0	83.0	0.0
94.0	83.0	0.0
91.0	83.0	1.0
90.0	89.0	1.0
90.0	83.0	0.0
90.0	83.0	1.0
90.0	83.0	1.0
90.0	83.0	0.0
89.0	84.0	1.0
89.0	84.0	1.0
89.0	83.0	1.0
90.0	83.0	1.0
93.0	83.0	0.0
90.0	83.0	1.0
91.0	84.0	0.0
90.0	85.0	1.0
89.0	83.0	1.0
90.0	83.0	0.0
90.0	89.0	1.0
90.0	92.0	1.0
91.0	92.0	3.0
90.0	95.0	0.0
90.0	92.0	1.0
100.0	93.0	1.0
101.0	92.0	1.0
102.0	93.0	1.0

Table 2: Tabela das 30 Inserções BB em microssegundos

semelhante ao das inserções. Os tempos de busca são mais altos nos cenários crescente e decrescente, variando de 17.0 a 94.0, o que reflete a degeneração da árvore nesses cenários. Em contraste, no cenário aleatório, os tempos de busca são significativamente menores, com muitos resultados próximos de 0.0. Isso demonstra que, sem o balanceamento adequado, a árvore BB pode ser ineficiente em buscas quando os dados são inseridos em ordem.

BB BUSCA		
CRESC.	DECRES	RANDOM
54.0	52.0	0.0
35.0	2.0	0.0
62.0	41.0	0.0
17.0	9.0	0.0
42.0	10.0	0.0
52.0	19.0	0.0
26.0	48.0	0.0
70.0	79.0	0.0
55.0	76.0	0.0
34.0	84.0	1.0
34.0	71.0	0.0
41.0	38.0	1.0
94.0	69.0	1.0
21.0	75.0	0.0
80.0	44.0	0.0
63.0	16.0	1.0
64.0	39.0	0.0
63.0	71.0	1.0
61.0	72.0	1.0
39.0	78.0	0.0
43.0	56.0	1.0
4.0	25.0	1.0
64.0	86.0	1.0
4.0	74.0	0.0
9.0	19.0	1.0
6.0	73.0	1.0
20.0	12.0	1.0
69.0	29.0	1.0
59.0	10.0	0.0
11.0	61.0	0.0

Table 3: Tabela das 30 Buscas BB em microssegundos

3.3 Inserções na Árvore AVL

A Tabela 4 apresenta os tempos de inserção da Árvore AVL para os três cenários. Observa-se uma estabilidade notável nos tempos, com todos os valores variando de 1.0 a 7.0. Isso reflete a característica de auto-balanceamento da árvore AVL, que impede sua degeneração e

mantém a profundidade da árvore baixa mesmo em cenários ordenados. O desempenho constante, independentemente da ordem dos dados, reforça a eficiência dessa estrutura para inserções rápidas.

AVL INSERÇÃO		
CRESC.	DECRES	RANDOM
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
0.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	0.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	0.0	1.0
1.0	0.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	0.0	1.0
1.0	1.0	1.0
1.0	0.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	2.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0
1.0	1.0	1.0

Table 4: Tabela das 30 Inserções AVL em microssegundos

3.4 Buscas na Árvore AVL

Na Tabela 5, os tempos de busca para a Árvore AVL mostram um padrão semelhante ao observado nas inserções, com tempos variando de 1.0 a 3.0 em todos os cenários. Esse comportamento consistente deve-se ao balanceamento automático da árvore, que mantém a profundidade controlada e permite buscas eficientes independentemente da ordem de inserção dos elementos.

AVL BUSCA		
CRESC.	DECRES	RANDOM
1.0	1.0	1.0
0.0	1.0	1.0
1.0	1.0	0.0
1.0	1.0	0.0
0.0	0.0	0.0
1.0	0.0	0.0
1.0	1.0	1.0
1.0	1.0	0.0
1.0	1.0	1.0
1.0	1.0	0.0
0.0	1.0	1.0
0.0	1.0	1.0
1.0	0.0	0.0
1.0	1.0	0.0
1.0	1.0	0.0
0.0	0.0	1.0
0.0	1.0	0.0
1.0	1.0	0.0
1.0	1.0	1.0
0.0	1.0	0.0
0.0	1.0	0.0
1.0	1.0	0.0
1.0	1.0	0.0
1.0	0.0	0.0
0.0	1.0	0.0
0.0	0.0	0.0
1.0	1.0	1.0
1.0	1.0	0.0
0.0	1.0	1.0
0.0	0.0	0.0

Table 5: Tabela das 30 Buscas AVL em microssegundos

3.5 AVL Inserção

Nesta tabela, são apresentados os resultados de médias e tempos totais para operações de inserção em uma árvore AVL, nas três categorias: dados crescentes, decrescentes e aleatórios.

AVL INSERÇÃO			
	Cresc.	Decres	Random
Média	1.0	0.96	1.03
Tempo	30.0	28.8	30.9

Table 6: Média e Tempo Total para AVL Inserção em Microssegundos

3.6 AVL Busca

A tabela abaixo exibe as médias e os tempos totais para buscas em uma árvore AVL, considerando dados em ordem crescente, decrescente e aleatória.

AVL BUSCA			
	Cresc.	Decres	Random
Média	0.67	0.83	0.53
Tempo	20.1	24.9	15.9

Table 7: Média e Tempo Total para AVL Busca em Microssegundos

3.7 BB Inserção

A tabela a seguir apresenta os resultados de médias e tempos totais para inserção utilizando uma Busca Binária (BB) nos três casos: ordem crescente, decrescente e aleatória.

BB INSERÇÃO			
	Cresc.	Decres	Random
Média	90.2	85.8	0.87
Tempo	2706	2574	26.1

Table 8: Média e Tempo Total para BB Inserção em Microssegundos

3.8 BB Busca

Nesta última tabela, são apresentados os resultados das médias e tempos totais para buscas em uma estrutura de Busca Binária (BB), com dados organizados em ordem crescente, decrescente e aleatória.

BB BUSCA			
	Cresc.	Decres	Random
Média	42.7	48.1	0.27
Tempo	1281	1443	8.1

Table 9: Média e Tempo Total para BB Busca em Microssegundos

4 Conclusão

O experimento teve como propósito o desenvolvimento de um programa em C para o cadastro, gerenciamento e busca de informações relacionadas a alunos, cursos, disciplinas, matrículas e notas, utilizando estruturas de dados como listas encadeadas e árvores binárias balanceadas (AVL). O objetivo era verificar a eficiência do uso dessas estruturas na organização e manipulação dos dados, além de permitir a avaliação do desempenho de operações de inserção e busca.

O propósito do experimento foi atingido com sucesso, permitindo a execução de todas as funcionalidades propostas, como o cadastro de novos alunos e disciplinas, inserção de matrículas e notas, e busca eficiente dos dados utilizando árvores AVL. Foram obtidos tempos de execução satisfatórios para as operações testadas, confirmando

que a estrutura de árvore balanceada trouxe uma melhora significativa na eficiência das buscas e inserções, especialmente à medida que o número de dados aumentava.

A execução do programa possibilitou a descoberta de que, em cenários com grande volume de dados, a utilização de árvores AVL oferece uma vantagem clara sobre outras estruturas de dados não balanceadas. Isso ficou evidente ao comparar o tempo de inserção e busca nas árvores, mostrando que a manutenção do balanceamento minimiza o impacto do aumento do número de elementos.

Com a realização dos experimentos, foi aprendido que a escolha adequada da estrutura de dados pode ter um impacto significativo no desempenho de operações comuns, como busca e inserção. Além disso, a implementação de árvores AVL exige atenção aos detalhes de balanceamento, mas oferece grande eficiência em situações com muitos dados, tornando-se uma ferramenta essencial para sistemas que requerem organização e acesso rápido a informações.